Toward Formally Verified TLS 1.3 Notarization: Joint Record Protection and Authenticated Release

Anonymous authors (research draft)

2026

Abstract

TLS notarization lets a client prove the provenance of selectively disclosed web data without server modification. Existing TLSNotary deployments target TLS 1.2, while TLS 1.3 changes the key schedule, record nonce construction, and encrypted-handshake boundary. We describe an implementation of bounded two-party TLS 1.3 application-record protection in which traffic keys remain secret-shared and server plaintext is released through a capsule gated by a jointly computed GCM tag. We separate assurance into symbolic protocol analysis, computational assumptions, machine-checked state invariants, and empirical validation. Tamarin establishes symbolic key secrecy, authenticated plaintext release, record-slot uniqueness, handshake/transcript agreement, Finished-before-application-secret installation, and bounded selective-disclosure equivalence. The record-layer properties are proved under ideal AEAD and MPC assumptions. Lean proves sequence and nonce invariants, and Kani checks the corresponding Rust functions over all 64-bit sequences and 96-bit IVs. The analysis also identifies a limitation: authenticated plaintext release does not by itself imply equality with the tag submitted to the verifier. A unified transition system proves provenance from negotiated handshake through presentation, while concrete refinement retains and independently rehashes the Rust transcript and proves SHA-384 circuit equivalence compositionally. Malicious-runtime, side-channel, and full computational MPC security remain outside the claim; consequently we do not claim a fully verified deployed system.

Introduction

TLS authenticates communication between a client and server but does not give the client a transferable proof of response provenance. TLSNotary addresses this gap by splitting TLS computations between a prover and verifier and later supporting selective disclosure (TLSNotary Project 2026). Related systems such as DECO also use collaborative cryptography to prove facts about TLS-protected web data without server modification (Zhang et al. 2020).

TLS 1.3 requires a new analysis boundary. Its record layer uses AEAD throughout, derives a per-record nonce by XORing a padded 64-bit sequence number with the static IV, and resets independently maintained read/write sequences on key changes (Rescorla 2018). In a notarization protocol, revealing the complete server application key to the prover destroys provenance: the prover could generate arbitrary ciphertext/tag pairs that appear to originate at the server.

This work makes five contributions:

1. A bounded joint AES-GCM application-record design that retains application keys and IVs as secret-shared MPC values.
2. An authenticated-release capsule that withholds the verifier's plaintext mask unless the prover can reconstruct the verifier's GCM tag share.
3. Reproducible Tamarin, Lean, and Kani artifacts with explicitly separated theorem boundaries.
4. Concrete transcript, suite-width, and SHA-384 circuit refinement proofs tied to the Rust implementation.
5. An empirical TLS 1.3 interoperability evaluation and an analysis of proof boundaries outside the verified implementation core.

Background and related work

TLS 1.3 specifies AEAD record protection, independent 64-bit read/write sequences, and nonce derivation from the static write IV (Rescorla 2018). GCM's security critically depends on key/IV uniqueness (Dworkin 2007), and its security bounds require care because flaws have appeared in earlier analyses (Iwata et al. 2012).

DECO established a prominent cryptographic approach to proving provenance of TLS data without trusted hardware or server changes (Zhang et al. 2020). TLSNotary uses MPC to split TLS session operations between a prover and verifier (TLSNotary Project 2026). The public TLSNotary documentation currently describes TLS 1.3 as future work, making the implementation and analysis here an experimental research contribution rather than a statement about an upstream release.

Tamarin models protocols as multiset-rewriting systems and proves trace or observational-equivalence properties in a symbolic model (Meier et al. 2013). Its results are relative to the supplied model and equational theory, and protocol verification is undecidable in general; automation may not terminate (The Tamarin Team 2026). We therefore report exact modeled assumptions and do not equate symbolic proof with computational or implementation security.

System and threat model

The prover controls the network-facing TLS client and may deviate arbitrarily. The verifier participates in MPC and retains secret shares. The initial release theorem considers a malicious prover and honest verifier. Parties do not collude, the server application key is uncompromised, randomness is fresh, and the invoked MPC and AEAD functionalities are ideal. A separate malicious- verifier confidentiality theorem is future work.

The evaluated profiles are TLS 1.3 with `TLS_AES_128_GCM_SHA256` or `TLS_AES_256_GCM_SHA384`, server authentication, bounded application records, and no 0-RTT, resumption, or `KeyUpdate` claim. Denial of service and side channels are outside the theorem boundary.

The suite restriction remains a deliberate fail-closed capability boundary: the production backend accepts the two AES-GCM suites above and rejects other TLS 1.3 suites before live key installation. The repository supplies the MPZ SHA-384 compression circuit and integrates secret-shared SHA-384/HMAC/HKDF, typed handshake/application material, 48-byte Finished, and live AES-256 read/write record epochs. The remaining generalization plan, including ChaCha20-Poly1305, is documented in `docs/research/tls13-cipher-suite-generalization.md`. The formal suite boundary also includes a separate Tamarin model for `TLS_AES_256_GCM_SHA384`: three lemmas verify SHA-384 Finished acceptance, Finished-before-application-secret ordering, and transcript-context binding. Concrete suite dispatch is additionally checked by Kani width-profile harnesses and genuine AES-256 interoperability tests.

The prover occupies several adversarial roles simultaneously:

Role	Status	Protected property
Network scheduler	Malicious	Record and transcript integrity
TLS client	Malicious	Server provenance
MPC leader/prover	Malicious	Key secrecy and authenticated release
Presenter	Malicious	Disclosure integrity
Verifier	Honest in the initial theorem	Confidentiality is future work
TLS server	Honest with respect to its authentication key	Certificate-bound server identity

The assurance boundary is intentionally explicit:

Component	Status in the stated claims
Tamarin equational theory and transition rules	Trusted as the symbolic model; theorems are relative to this model
Tamarin, Maude, Lean, and Kani checkers	Trusted toolchain for the reported machine-checked results
TLS 1.3 specification and SHA-256/GCM assumptions	Cryptographic and protocol assumptions
MPZ VM, MPC circuits, Rust parser, and serialization	Implementation under test; not assumed correct by the symbolic proofs
Honest verifier, uncompromised server traffic secret, and fresh randomness	Initial theorem assumptions
Malicious prover and network scheduling	Adversarially controlled in the record-layer model
Malicious verifier, side channels, crashes, and denial of service	Outside the current theorem boundary

Construction

Application traffic keys and IVs remain references in the MPC VM. Typed read and write epochs own independent sequence numbers. A record consumes its sequence immediately before AEAD evaluation; exhaustion returns an error instead of wrapping. The 96-bit nonce is `IV xor (0^32 || uint64(sequence))`.

For incoming records, both parties mask the candidate plaintext. The verifier releases its mask in an HMAC-based capsule keyed by its secret GCM tag share. The complete tag is the XOR of leader and follower shares. Given the public record tag and its own share, the prover derives the candidate follower share; only the authentic candidate opens and validates the capsule under the stated PRF and share-uniformity assumptions. The detailed game sequence and proposed hardening are maintained in the accompanying computational-proof artifact.

Every record is identified by `RecordId = (session_id, direction, generation, sequence)`. The session is derived from the authenticated handshake context. The identifier is carried through MPC allocation, release derivation, commitments, and presentation; the unified Tamarin model proves that it cannot transfer between sessions.

Concrete implementation

The implementation is organized as a layered Rust pipeline rather than as a single monolithic TLS proof. The TLS-facing MPC-TLS crate owns handshake state, transcript messages, epochs, and record orchestration. The `tlsn-hmac-sha256` component crate owns the PRF circuits and exposes VM memory references to their outputs. The record layer consumes those references through the MPZ VM; it does not receive a decoded application key.

The implementation boundary is distributed across four principal layers:

- TLS syntax and state machine (`crates/tls/client`): parses handshake messages, retains their exact wire encoding, enforces TLS ordering, and invokes backend callbacks.
- Collaborative handshake (`crates/mpc-tls/src`): agrees protocol version and suite, validates transcript callbacks, executes the joint key schedule, and installs typed epochs.
- Secret-shared primitives (`crates/components/hmac-sha256/src/tls13`): implements SHA-256/SHA-384, HMAC, HKDF-Extract, HKDF-Expand-Label, Finished, and typed traffic-key views.
- Records and attestations (`crates/mpc-tls/src/record_layer` and `crates/tlsn/src`): reserves nonces, runs joint AES-GCM, gates plaintext release, assigns session-bound record identities, and commits disclosures.

The leader is the network-facing prover. The follower is the verifier-side MPC participant. They execute the same protocol version, cipher-suite, transcript, and epoch transitions, but neither receives the complete

application traffic key. Public TLS values—record headers, ciphertext, server tag, certificate material, and Finished verify data—may be decoded where the protocol requires them. Secret traffic material remains represented by MPZ memory references.

Handshake and key schedule

For both negotiated profiles, the key-schedule state follows the TLS 1.3 stages:

1. the key-exchange component produces the pre-master secret;
2. `HandshakeSecrets` computes the handshake secret and the client/server handshake traffic secrets with HKDF-Extract and HKDF-Expand-Label;
3. the ClientHello transcript hash is supplied before handshake key material is completed;
4. Finished verification precedes application-secret installation; and
5. the final transcript hash drives the client/server application traffic secrets and their `key` and `iv` labels.

The schedule is stateful. Invalid transitions, repeated hash installation, suite/hash-width confusion, and sequence exhaustion return errors. Application keys are retained as typed 16- or 32-byte MPZ views with 12-byte IV views; they remain VM references and are not decoded.

Negotiation and suite profiles The TLS parser reports the selected `CipherSuite` to the MPC backend before any suite-dependent transcript value is accepted. Leader and follower bind that message exactly once to a `Tls13SuiteProfile`. The AES-128 profile fixes a 32-byte SHA-256 digest and 16-byte traffic key; the AES-256 profile fixes a 48-byte SHA-384 digest and 32-byte traffic key. A later attempt to rebind the suite, present a digest of the other width, or install keys of the wrong width fails before record processing. Unsupported suites are rejected at this same boundary, so capability negotiation is fail closed.

This explicit profile prevents a class of cross-suite state confusion in which a length-delimited digest would otherwise be dispatched solely by its byte length. Length remains checked, but it is checked against the already-bound suite rather than used to infer the suite.

Exact transcript refinement `HandshakeHash` feeds the ordinary TLS digest while also retaining an audit copy of every encoded handshake byte. The copy follows the same update points as the digest, including the synthetic `message_hash` transition used after a HelloRetryRequest. At ServerHello, CertificateVerify, server Finished, and client Finished, the TLS state machine supplies both the digest and the exact encoded prefix to MPC-TLS.

The leader recomputes the digest from those bytes with the hash selected by the negotiated suite before it sends the callback to the follower. The follower then performs the same recomputation independently. CertificateVerify and Finished processing cannot advance if either recomputation differs, if the digest width is wrong, or if the callback arrives under a different suite profile. Thus the MPC key schedule does not merely trust an opaque hash chosen by the network-facing party.

This refinement covers the bytes entering the TLS transcript and their use at the MPC callback boundary. It does not prove the complete X.509 parser, certificate path builder, or every branch of the surrounding TLS client.

Secret-shared derivation The key schedule mirrors RFC 8446's early, handshake, and master-secret stages. HKDF labels are serialized as `uint16(output_length) || uint8(label_length) || "tls13 " || label || uint8(context_length) || context`. Bounds are checked before the narrowing integer conversions. Traffic-secret expansion produces typed subviews for `key`, `iv`, and `finished`; it does not decode the containing secret to create those views. Client and server directions receive distinct label domains and distinct epoch objects.

Finished is computed jointly as HMAC over the public transcript hash using the secret-shared Finished key. Only the public verify-data output is decoded. The application allocator is reachable only after server Finished acceptance, and the post-Finished transcript hash is installed once to derive application traffic secrets.

The SHA-384 circuit added for AES-256 work

The pinned MPZ distribution contains SHA-256 circuit data but no SHA-384 compression circuit. We therefore construct the SHA-384/SHA-512 compression circuit locally in `sha384_circuit.rs`. It contains the eight SHA-384 initial state words, all 80 SHA-512 round constants, 64-bit modular additions, rotations, `Ch` and `Maj`, and the message schedule. `sha384_vm.rs` adapts the static circuit to MPZ VM calls; `sha384.rs` supplies streaming block handling, TLS-compatible 128-bit length padding, and serialization of the first six 64-bit state words into 48 bytes.

The circuit is not a cleartext fallback. `hmac384.rs` computes secret-shared ipad/opad partial states and HMAC-SHA384; `hkdf_extract384.rs` accepts one or more secret-shared IKM vectors (including the hybrid key-exchange case), and `hkdf384.rs` implements the TLS 1.3 `HkdfLabel` expansion. Typed 32-byte key and 12-byte IV views are obtained from the 48-byte SHA-384 expansion without decoding the surrounding secret. Handshake and application modules then derive `c hs traffic`, `s hs traffic`, `c ap traffic`, and `s ap traffic`, followed by the `key`, `iv`, and `finished` labels. Finished verification is an HMAC over the 48-byte transcript hash and returns only public verify data.

The implementation has separate reference tests for compression, multi-block streaming, HMAC, HKDF-Extract, HKDF-Expand-Label, handshake material, application material, and Finished output. These tests execute both MPZ parties and compare the result with `sha2`, `hmac`, and clear TLS 1.3 oracle functions.

For formal refinement, the generated circuit and its verification instantiations share the same generic Boolean primitives, message schedule, and 80-round compression function. Kani proves wrapping addition, `Ch`, `Maj`, and all sigma functions for arbitrary 64-bit inputs, then proves the shared compression function equal to `sha2::compress512` for every block and chaining state. SHA-384 uses that compression function with its specified initial value and six-word output truncation.

The serialized production artifact `data/sha384.bin` is generated from this same circuit builder. Reproducibility checks regenerate the artifact and compare it byte for byte, preventing a proved source generator from silently diverging from the circuit actually loaded by `sha384_vm.rs`.

Epochs, records, and release

`WriteEpoch<K, I>` and `ReadEpoch<K, I>` own the traffic key, IV, generation, and next sequence number. Reservation occurs immediately before encryption or authentication, including failed authentication attempts. The nonce helper XORs the big-endian sequence into the low 64 bits of the 96-bit IV. The AES-128 and AES-256 use the joint GCM record implementation and authenticated- release capsule with the same AAD, padding, sequence, and nonce rules.

The SHA-384 work is connected to MPC-TLS through typed handshake and application epoch slots. Both parties execute the allocator, and the state level Finished callback decodes only the resulting public 48-byte verify data. Inter-party transcript and Finished messages carry length-delimited digests and the exact encoded transcript. Leader and follower independently recompute each callback digest under the negotiated profile before accepting it. Live suite selection dispatches 48-byte callbacks and AES-256 record epochs together.

Record processing Each directional epoch owns `(key, iv, generation, next_sequence)`. The caller cannot supply an arbitrary sequence number. Reserving a record returns the current value and advances the epoch exactly once; `u64::MAX` is rejected rather than wrapped. The record nonce is constructed as required by RFC 8446 by XORing the big-endian sequence number into the low eight bytes of the static 12-byte IV. A key transition installs a fresh epoch and resets only that epoch's sequence.

For an outgoing application record, the plaintext and inner content type are padded according to TLS 1.3, encrypted with the joint CTR path, and authenticated with GHASH over the five-byte TLSCiphertext header and ciphertext. For an incoming record, the parties allocate the same nonce slot, jointly decrypt a candidate plaintext, compute additive shares of the expected GCM tag, and run the authenticated-release step. A failed authentication attempt still consumes its sequence slot, preventing retry-induced nonce reuse.

AES-128 and AES-256 share this orchestration. The suite profile chooses the typed key width and preallocated AES circuit, while nonce construction, AAD, padding, sequence ownership, and release semantics remain common.

Session and presentation binding A `SessionId` is derived from authenticated handshake material using a domain-separated SHA-256 encoding. TLS 1.3 binds the transcript hash and ephemeral key; the retained TLS 1.2 compatibility path binds its randoms and key material. Every record receives `RecordId = (SessionId, direction, generation, sequence)`. Record construction checks that supplied identifiers match the transcript's session and expected direction.

The same identifiers are included in authenticated-release context, plaintext-hash commitment prefixes, secrets handed to the attestation API, and presentation verification. The commitment prefix also includes the disclosure direction, byte ranges, and ordered record identifiers before the plaintext and blinder are hashed. Consequently, identical plaintext at the same sequence in two sessions does not produce an interchangeable proof object.

Failure behavior and bounded resources The implementation preallocates a configured number of handshake and application record circuits before online execution. Exceeding that bound, using the wrong epoch, repeating a one-shot transition, exhausting a sequence, or observing inconsistent leader/follower message counts returns an error. Unused preallocations are assigned and committed during finalization so that both VM parties complete the same circuit schedule. This is important for correctness and deadlock avoidance, but it is not a denial-of-service theorem: an adversarial peer can still abort or delay the protocol.

Formal analysis

No single tool proves the complete claim. We use a refinement ladder in which each layer has a deliberately narrower obligation:

1. Tamarin proves adversarial protocol traces, ordering, secrecy events, provenance, disclosure binding, and cross-session non-transferability in symbolic transition systems.
2. Lean proves algebraic and structural specifications for epoch transitions, nonce injectivity, HKDF label layout, suite widths, and key-schedule domains.
3. Kani model-checks the corresponding concrete Rust state operations and byte-level encoders for arbitrary bounded inputs, and proves the SHA-384 Boolean construction compositionally.
4. Two-party executable tests compare MPC outputs with independent clear cryptographic implementations and exercise adversarial failure cases.
5. End-to-end fixtures and external servers establish that the composed code actually negotiates and transports TLS 1.3 records.

The layers are complementary. A Tamarin theorem cannot establish that a Rust encoder emitted the intended bytes; a Kani harness for a nonce helper cannot establish session provenance; and interoperability cannot establish security. The claim below uses each result only at its stated boundary.

Implementation obligation	Artifact	Machine-checked statement
Read/write sequence ownership	`Tls13Epoch.lean`; Kani harnesses in MPC-TLS	Successful reservation advances once, preserves generation, and rejects exhaustion
TLS nonce construction	Lean plus Kani over `make_tls13_nonce`	Fixed-IV nonce mapping is injective over all 64-bit sequences
Negotiated width profile	`Tls13Sha384.lean`; Kani over `Tls13KeyState`	SHA-256/16-byte and SHA-384/32-byte profiles cannot be confused

Implementation obligation	Artifact	Machine-checked statement
HKDF label bytes	`Tls13HkdfLabel.lean`; Kani over `make_hkdf_label`	Prefix, lengths, bounds, labels, and arbitrary transcript contexts match the concrete encoding
Transcript callbacks	Concrete Kani/unit refinement checks	Exact encoded bytes recompute to the callback digest under the negotiated suite
SHA-384 compression	Shared generic circuit/native wiring plus Kani	Every Boolean primitive and the full 80-round compression refine the independent SHA-512 reference for arbitrary inputs
Protocol provenance	Unified Tamarin model	Accepted presentation has preceding authenticated handshake, record, release, and attestation events
Cross-session binding	Unified Tamarin model plus Rust tests	The same full record identifier cannot be accepted under distinct sessions

Symbolic model

The record-layer Tamarin model idealizes AEAD plaintext and authentication, private MPC evaluation, and additive tag-share reconstruction. It gives the adversary network control and the leader tag share but withholds the follower share. Five required lemmas close automatically:

Lemma	Result	Boundary
Honest execution	Verified	Existence of one releasable record
Application-key secrecy	Verified	Absent explicit key compromise
Authenticated plaintext release	Verified	Ideal AEAD/MPC; malicious prover
Nonce tuple uniqueness	Verified	Modeled read-slot state
Read-slot single use	Verified	One verification attempt per slot

Finding 1 — Authenticated release does not imply submitted-tag agreement

The original stronger tag-agreement lemma was falsified. A prover retaining a genuine tag for a genuine ciphertext can use it to open a capsule after submitting another tag. The final theorem therefore establishes that released plaintext corresponds to a server-authenticated ciphertext under some valid tag, not equality with the submitted tag.

The attack trace is:

```
server emits (C, T_valid)
prover submits (C, T_fake) but retains T_valid
capsule opens under T_valid
plaintext is authentic, but T_submitted != T_valid
```

This is a positive formal-methods result: verification found and invalidated an intuitively stronger claim before publication.

A second Tamarin model covers the symbolic handshake/transcript boundary. It closes five additional lemmas for handshake executability and agreement, application-epoch agreement, presentation agreement, and server-identity binding. The model treats the concrete TLS 1.3 HKDF and certificate parser as abstract interfaces; it therefore does not establish those implementation details.

A third model isolates the TLS 1.3 key-schedule boundary. It represents HKDF-Extract and HKDF-Expand-Label as private symbolic constructors, derives the traffic secret using a labeled transcript context, and requires Finished verification before application-secret installation. Five lemmas close: an executable Finished path, emission-before-acceptance, acceptance before application-secret installation, transcript-context binding, and exact `c ap traffic` label/context formation. These are protocol-boundary claims; they do not replace test-vector or bit-level refinement proofs for the production HKDF code.

A fourth, deliberately minimal, diff model checks that changing unrevealed bytes while keeping the public projection fixed is observationally invisible. This is a boundary test for the disclosure interface, not a proof of the production circuit or serialization format.

A fifth model covers the SHA-384/AES-256 suite boundary. It verifies that a SHA-384 Finished event precedes acceptance and application-secret installation, and that the installed application context is bound to the SHA-384 transcript. The concrete negotiated suite profile and live interoperability tests connect that symbolic boundary to the enabled AES-256 production path.

End-to-end composition theorem

The intended end-to-end theorem is:

```
HandshakeAgreement
AND FinishedAcceptance
AND ApplicationSecretInstallation
AND EpochBinding
AND RecordAuthentication
AND DisclosureBinding
IMPLIES PresentedByteHasTLSServerProvenance
```

The unified `tls13_end_to_end.spthy` transition system proves this implication across handshake, Finished, application-secret installation, epoch use, record authentication, release, and presentation. It additionally proves that the presented session is handshake-derived and that a full `RecordId` cannot transfer between sessions. This remains a symbolic composition theorem; the concrete refinements below connect selected Rust boundaries but do not turn it into a whole-runtime theorem.

State and implementation invariants

Lean proves that the abstract epoch reservation operation returns the owned sequence, advances it exactly once, preserves the key generation, rejects exhaustion, and cannot return a wrapped sequence. It also proves nonce injectivity for a fixed IV using XOR cancellation.

A companion Lean specification proves the length and `tls13` prefix framing of `HkdfLabel`, matching the Rust encoder's structural obligation. Kani then checks the concrete encoder for every possible 32-byte and 48-byte transcript context, including its narrowing bounds. The Lean SHA-384 specification proves the digest, key, IV, Finished, and label-domain widths and that the client, server, handshake, application, key, IV, and Finished labels are distinct.

Four Kani harnesses model-check the actual Rust read/write reservation methods and nonce function for all `u64` values and all 96-bit IVs. These checks connect the stated local invariants to the implementation but do not constitute a whole-program refinement proof.

Additional Kani harnesses check that suite negotiation fixes the only accepted hash/key-width pair and that it cannot be rebound. For SHA-384, the actual circuit builder implements generic bit and word traits. Instantiating those traits with Boolean values lets Kani prove ripple-carry wrapping addition, choice, majority,

and the four SHA-512 sigma functions over all 64-bit inputs. Instantiating the shared schedule and compression wiring with native `u64` words then lets Kani compare all 80 rounds with `sha2::compress512` for an arbitrary block and chaining state. The circuit instantiation uses the same generic functions, leaving MPZ's XOR, AND, inversion, serialization, and VM execution semantics in the trusted implementation base.

Concrete-to-symbolic refinement boundary

The end-to-end symbolic theorem consumes events such as `HandshakeAuthenticated`, `ApplicationEpoch`, `RecordAuthenticated`, `PlaintextReleased`, and `PresentationAccepted`. Concrete code does not emit a Tamarin trace at runtime, so the connection is established by matching guarded state transitions and data carried across them:

- suite binding and independent transcript recomputation guard handshake and Finished callbacks;
- application epoch installation occurs only after the accepted Finished callback and carries suite-typed key/IV references;
- epoch-owned direction, generation, and sequence form the concrete `RecordId` used by record authentication;
- authenticated release carries that record context; and
- commitment construction and presentation verification retain the same session and record identifiers.

This is stronger than unrelated local tests because the same values cross all concrete boundaries and the unified symbolic theorem reasons about their full ordering. It is weaker than verified compilation or a whole-program refinement theorem: we trust the reviewed correspondence between Rust transitions and symbolic events, plus the toolchain and runtime components listed in the trusted computing base.

Computational argument

The current proof draft uses game hops replacing the domain-separated HMAC-SHA256 families with random functions, conditions on distinct uniform 128-bit follower shares, replaces the release pad with a one-time pad, and bounds wrong-key commitment acceptance. For `q` slots and `q_o` opening attempts, the target bound includes HMAC PRF advantage, GCM forgery advantage, MPC failure advantage, `q(q-1)/2^129` share collisions, `q_o/2^128` key guessing, and `q_o/2^256` commitment acceptance. This is not yet an externally reviewed reduction.

Implementation and evaluation

The implementation is in Rust and builds on MPZ components for garbled circuits, share conversion, and finite-field operations. AES/J0 circuits are preallocated for bounded application records. Unused allocations are assigned and committed during finalization so both VM parties terminate consistently.

Current validation includes unit tests, an end-to-end TLS 1.3 notarization fixture, a five-case certificate/client-authentication matrix, and interoperability with nginx, Apache httpd, Caddy, and OpenSSL `s_server`. On the reference arm64 Mac17,6 (18 logical CPUs, Rust 1.97.1), the existing Criterion harness reports 20.824 ms median-equivalent midpoint for the normal TLS 1.3 key schedule and 15.646 ms for the reduced schedule (10 samples per mode). These are indicative local measurements, not a cross-machine performance claim; the benchmark command and environment are recorded for reproduction.

The Docker-backed interoperability matrix was rerun on the verification branch. All five cases passed: nginx RSA, nginx ECDSA, Apache RSA, Caddy RSA, and OpenSSL `s_server`. The exact command is `./formal/interop.sh`; the focused fixture and core validation are run by `./formal/validate.sh`.

The implementation-facing reproducibility split is intentional: `cargo test -p tlsn-hmac-sha256 --lib` exercises 36 component tests, including 12 focused SHA-384/reference tests; `cargo test -p tlsn-mpc-tls` exercises typed epochs, AES-256 record round trips, SHA-384 epoch installation, and the public Finished callback; `formal/validate.sh` adds the TLS 1.3 fixture; and `formal/verify.sh` adds the focused integra-

tion tests before running the theorem checkers. This separates circuit/reference equivalence evidence from protocol interoperability evidence and from symbolic proof evidence.

Claim–evidence matrix

Claim	Evidence now	Missing evidence
Keys are not intentionally decoded	Code audit; symbolic secrecy	Whole-program information-flow/refinement proof
Authenticated release implies modeled server provenance	Unified Tamarin theorem and concrete session/record binding	Computational malicious-security reduction for the full MPC runtime
Epoch sequences do not wrap or repeat locally	Lean and Kani	Concurrent whole-program refinement
Fixed-IV nonce derivation is injective	Lean and Kani	Circuit/reference equivalence
SHA-384 compression refines SHA-512 compression	Shared circuit/native wiring; arbitrary-input Kani primitive and compression proofs; MPZ tests	MPZ primitive gate semantics are trusted
SHA-384 handshake/application key widths are preserved	Typed VM views, Lean width theorem, and concrete suite-profile Kani proofs	Whole-program runtime noninterference
SHA-384 Finished output is public while its key stays secret-shared	Two-party MPC-TLS callback test	Full malicious-party composition
Full proof transcript is authentic	Unified provenance theorem, exact Rust transcript retention, independent leader/follower rehashing	X.509 parser and full-runtime refinement
Selective disclosure preserves the public projection	Minimal Tamarin observational-equivalence model	Production leakage function and serialization refinement

The interoperation result is evidence of compatibility, not a security proof; the server implementations are honest test peers and do not exercise the malicious-party experiments.

Limitations and research agenda

The strongest missing result is a composable malicious-security theorem for the complete MPZ protocols and release capsule. The current proofs do not cover X.509 implementation correctness, malicious-verifier privacy, side channels, concurrency, crashes, denial of service, or whole-runtime noninterference. AES and GHASH have executable circuit/reference equivalence evidence; unlike the SHA-384 compression refinement above, they do not yet have arbitrary-input machine proofs. The tag-share capsule is nonstandard and should preferably be replaced by an explicitly context-bound release key derived inside MPC.

Accordingly, the correct claim is a working bounded TLS 1.3 notarization implementation with machine-checked protocol, provenance, session-binding, state, suite, transcript, HKDF-label, and SHA-384 compression layers—not a proof of the complete deployed software stack against every adversary.

Reproducibility

The repository contains the Tamarin theories, Lean specification, Kani harnesses, and `formal/verify.sh`, which fails unless every required lemma is reported as verified. On the reference machine, the

wrapper runs Tamarin 1.12.0, Maude 3.5.1, Lean 4.32.2, and Kani 0.67. The executable validation suite reports 36 MPC tests, 36 HMAC/component tests, 3 core configuration tests, and the TLS 1.3 integration fixture; the explicit Docker interoperability suite passes nginx (RSA and ECDSA), Apache (RSA), Caddy (RSA), and OpenSSL `s_server`. These counts are reproducibility anchors, not a claim of exhaustive coverage. The full command matrix, including the RFC-derived TLS 1.3 HKDF vectors in `crates/components/hmac-sha256/src/kdf/expand.rs`, is recorded in `formal/REPRODUCIBILITY.md`. The symbolic-to-Rust mapping and the unproved refinement obligations are listed in `formal/REFINEMENT_BOUNDARY.md`.

Conclusion

TLS 1.3 notarization can retain the central TLSNotary provenance invariant by keeping application keys secret-shared and gating plaintext release on joint record authentication. Layered verification exposed both useful positive results and a subtle boundary between plaintext authenticity and exact tag agreement. Completing computational malicious-security composition, whole- runtime noninterference, X.509 refinement, side-channel analysis, and malicious-verifier privacy remains necessary before claiming verification of the entire deployed system.

References

Dworkin, Morris. 2007. *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. NIST SP 800-38D. National Institute of Standards; Technology. https://doi.org/10.6028/NIST.SP.800-38D.

Iwata, Tetsu, Keisuke Ohashi, and Kazuhiko Minematsu. 2012. "Breaking and Repairing GCM Security Proofs." *Advances in Cryptology – CRYPTO 2012*.

Meier, Simon, Benedikt Schmidt, Cas Cremers, and David Basin. 2013. "The TAMARIN Prover for the Symbolic Analysis of Security Protocols." *Computer Aided Verification*.

Rescorla, Eric. 2018. *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446. https://doi.org/10.17487/RFC8446.

The Tamarin Team. 2026. *Tamarin Prover Manual*. https://tamarin-prover.com/manual/.

TLSNotary Project. 2026. *TLSNotary Documentation*. https://tlsnotary.org/docs/intro/.

Zhang, Fan, Deepak Maram, Harjasleen Malvai, Steven Goldfeder, and Ari Juels. 2020. "DECO: Liberating Web Data Using Decentralized Oracles for TLS." *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 1919–38. https://doi.org/10.1145/3372297.3417239.